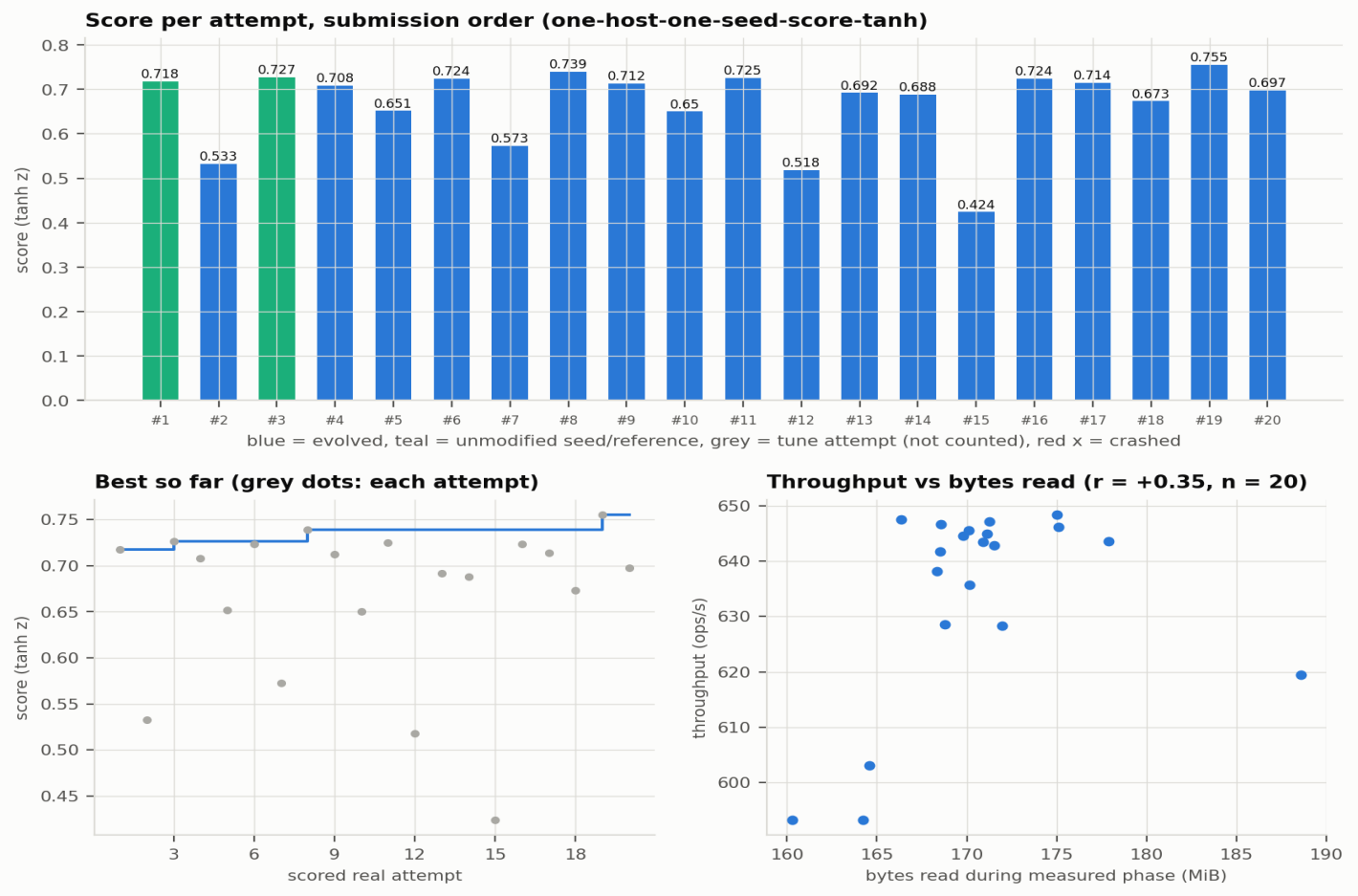
CORAL run 2026-10-20_173031 — one-host-one-seed-score-tanh

S3-FIFO is the only seed; score = tanh-squashed weighted z-score vs the vanilla-kernel no-op baseline (throughput w=2.0, bytes read 0.5, refaults 0.25). Evaluated on CloudLab host pc309.emulab.net (2-core, ~2 GB Xeon; absolute throughput is not comparable with the earlier c220g1 runs or with the other hosts).

real attempts	20 (cap 20) | scored 20, crashed 0 | tune 0
stop reason	max_real_attempts
wall time (agent logs)	0.64 h
best attempt	#19 fefe1ef1 score 0.755 (throughput 647.6 ops/s) — agent-evolved
best evolved	#19 fefe1ef1 score 0.755 (throughput 647.6 ops/s)
unmodified seed (s3fifo)	#1: 0.718, #3: 0.727
LLM usage	38.27 USD, 46k output tokens, 93 turns in 21 sessions; gateway requests 152 {400: 43, 200: 109}
tanh baseline (no-op, n=8)	throughput 549.2 ± 100.9 ops/s on this host



All attempts (submission order)

#	commit	class	score	tput ops/s	read MiB	refaults	wall s	design / title
1	7c8aad1f	real =s3fifo	0.718	645.0	171	18634	40	baseline: unmodified S3-FIFO seed (comment-only change)
2	c3201817	real	0.533	593.2	160	16378	40	S3-FIFO + scan list: per-TID sequential-run detector (run>=48 consecutive adds, gap<=32) flags scanner thread; its folios go to a scan FIFO evicted fi
3	76284f7b	real =s3fifo	0.727	645.6	170	18428	40	noise check: seed S3-FIFO re-run (comment-only)
4	22cf62a9	real	0.708	642.9	172	18683	41	sweep: S3-FIFO small queue target cache/15 -> cache/10
5	6bc4912b	real	0.651	643.6	178	20330	40	seed S3-FIFO + per-TID scanner detector (run>=48 adds, gap<=32); only change: folio_accessed ignores scanner-thread touches (no freq bump), queues/gho
6	d3cd4036	real	0.724	647.2	171	18676	40	seed S3-FIFO with ghost map sized 2x cache (was 1x): more readmissions straight to main
7	92387d00	real	0.573	603.1	165	17272	40	scan-list policy (c3201817) with scan floor raised cache/50 -> cache/8: hypothesis is HDD-like seek-bound I/O, eval2 lost throughput because scan read
8	f5ee59d6	real	0.739	646.6	169	18205	40	diagnostic: scan-first policy (c3201817) with strict scanner detector run>=200 adds, gap<=8, credit 1024 — tests whether evals 2/7 regressed from misc
9	32fb4c38	real	0.712	638.2	168	18068	40	scan-first + detector sweep: run>=100 (was 200), gap<=8, credit 1024 — more scan coverage vs eval8 best (646.6 ops/s)
10	bcb23062	real	0.650	628.3	172	18841	40	eval8 best (strict scan detector run>=200 gap<=8) + ghost map 2x cache (eval6 gave +0.3% on seed)
11	490b48ea	real	0.725	644.6	170	18429	40	reproducibility check: exact rerun of eval8 best f5ee59d (strict scan detector), comment-only change
12	f8d836f6	real	0.518	593.2	164	17173	40	structural: S3-FIFO small queue kept, main queue eviction switched from CLOCK (4-pass iterate) to sampled LFU (sample 16, evict min freq, no decrement
13	d3158df0	real	0.692	635.7	170	18458	41	sweep other side: S3-FIFO small queue target cache/15 -> cache/30 (1/10 was neutral, 642.9)
14	6190d4d5	real	0.688	646.2	175	19623	41	structural attempt 1/3 on scan-favoring (H2: throughput limited by scan speed): loose detector (run>=48,gap<=32) scanner folios admitted straight to m
15	a4696ed4	real	0.424	619.4	189	22910	40	structural attempt 2/3 on scan-favoring: scanner folios admitted to main with freq=3 (cap) — strongest scan retention; if throughput still ~645, H2 (s
16	485600de	real	0.724	641.7	169	18073	41	seed S3-FIFO with freq cap 3 -> 7: hot GET pages survive more CLOCK sweeps in main (scan-favoring lane closed: eval15 refuted H2)
17	94d68cf6	real	0.714	643.5	171	18600	40	scan-quota (paper-inspired, but our total-throughput metric needs scans): scan list evicted first only when it exceeds 25% of cache; detector run>=100
18	ed92edd3	real	0.673	628.6	169	18134	40	seed S3-FIFO: ghost-hit readmissions start at freq=1 instead of 0 (survive one main CLOCK sweep)
19	fefe1ef1	real	0.755	647.6	166	17599	40	peak test (opposite of eval18): ghost hits go to small queue with freq=1 (one more access promotes) instead of straight to main
20	0a567def	real	0.697	648.4	175	19481	41	direction test from eval19 best: ghost hits -> small with freq=0 (ghost gives no advantage at all) vs freq=1

Best evolved policy: #19 fefe1ef1 — score 0.755

Title: peak test (opposite of eval18): ghost hits go to small queue with freq=1 (one more access promotes) instead of straight to main

Closest available seed by line similarity: s3fifo 99%. Lines that differ from s3fifo are highlighted below.

```
// ============================================================================
// SECTION: BPF KERNEL CODE
// ============================================================================
// EVOLVE-BLOCK-START
// S3-FIFO (ghost hits -> small, freq 1): a small FIFO (~10% of the cache) admits new folios; a folio re-accessed while in
// the small queue is promoted to the main queue, otherwise evicted. The main queue is
// CLOCK-like with a frequency counter capped at 3. A ghost map remembers recently evicted
// folios so a quick re-reference is admitted straight into main.
#include "vmlinux.h"
#include <bpf/bpf_helpers.h>
#include <bpf/bpf_tracing.h>
#include <bpf/bpf_core_read.h>

#include "cache_ext_lib.bpf.h"
#include "dir_watcher.bpf.h"

char _license[] SEC("license") = "GPL";

#define ENOENT      2  /* include/uapi/asm-generic/errno-base.h */
#define INT64_MAX   (9223372036854775807LL)

// Set from userspace. In terms of number of pages.
// TODO: change

//#define CACHE_SIZE (((1ull << 30) * 2) / 4096)
#define CACHE_SIZE (((1ull << 20) * 200) / 4096)
const volatile size_t cache_size = 0;

struct folio_metadata {
    s64 freq;
    bool in_main;
};

struct ghost_entry {
    u64 address_space;
    u64 offset;
};

struct {
    __uint(type, BPF_MAP_TYPE_HASH);
    __type(key, u64);
    __type(value, struct folio_metadata);
    __uint(max_entries, 4000000);
} folio_metadata_map SEC(".maps");

struct {
    __uint(type, BPF_MAP_TYPE_LRU_HASH);
    __type(key, struct ghost_entry);
    __type(value, u8);
    //__uint(max_entries, CACHE_SIZE); // TODO: change
    __uint(map_flags, BPF_F_NO_COMMON_LRU);  // Per-CPU LRU eviction logic
} ghost_map SEC(".maps");

static u64 main_list;
static u64 small_list;

/*
 * This is an approximate value based on what we choose to evict, not what is
 * actually evicted.
 */
static s64 small_list_size = 0;
static s64 main_list_size = 0;

static inline bool is_folio_relevant(struct folio *folio) {
    if (!folio || !folio->mapping || !folio->mapping->host)
        return false;

    return inode_in_watchlist(folio->mapping->host->i_ino);
}

static inline struct folio_metadata *get_folio_metadata(struct folio *folio) {
    u64 key = (u64)folio;
    return bpf_map_lookup_elem(&folio_metadata_map, &key);
}

/*
 * Check if a folio is in the ghost map and delete the ghost entry.
 * We only check if an element is in the ghost map on inserting into the cache.
 * Relies on bpf_map_delete_elem() returning -ENOENT if the element is not found.
 */
static inline bool folio_in_ghost(struct folio *folio) {
    struct ghost_entry key = {
        .address_space = (u64)folio->mapping->host,
        .offset = folio->index,
    };
    // TODO: handle non-ENOENT errors
    return bpf_map_delete_elem(&ghost_map, &key) != -ENOENT;
}

s32 BPF_STRUCT_OPS_SLEEPABLE(evo_policy_init, struct mem_cgroup *memcg)
{
    main_list = bpf_cache_ext_ds_registry_new_list(memcg);
    if (main_list == 0) {
        bpf_printk("cache_ext: init: Failed to create main_list\n");
        return -1;
    }
    bpf_printk("cache_ext: Created main_list: %llu\n", main_list);

    small_list = bpf_cache_ext_ds_registry_new_list(memcg);
    if (small_list == 0) {
        bpf_printk("cache_ext: init: Failed to create small_list\n");
        return -1;
    }
    bpf_printk("cache_ext: Created small_list: %llu\n", small_list);

    return 0;
}

static s64 bpf_s3fifo_score_main_fn(struct cache_ext_list_node *a) {
    if (!folio_test_uptodate(a->folio) || !folio_test_lru(a->folio))
        return INT64_MAX;

    if (folio_test_dirty(a->folio) || folio_test_writeback(a->folio))
        return INT64_MAX;

    struct folio_metadata *data = get_folio_metadata(a->folio);
    if (!data) {
        bpf_printk("cache_ext: score_fn: Failed to get metadata\n");
        return INT64_MAX;
    }
    s64 freq = __sync_sub_and_fetch(&data->freq, 1);
    if (freq < 0)
        data->freq = 0;

    return freq;
}

static int bpf_s3fifo_score_small_fn(int idx, struct cache_ext_list_node *a)
{
    if (!folio_test_uptodate(a->folio) || !folio_test_lru(a->folio))
        return CACHE_EXT_CONTINUE_ITER;

    if (folio_test_dirty(a->folio) || folio_test_writeback(a->folio))
        return CACHE_EXT_CONTINUE_ITER;

    struct folio_metadata *data = get_folio_metadata(a->folio);
    if (!data) {
        bpf_printk("cache_ext: score_fn: Failed to get metadata\n");
        return CACHE_EXT_CONTINUE_ITER;
    }

    // Move to main list if freq > 1
    if (data->freq > 1) {
        data->in_main = true;
        return CACHE_EXT_CONTINUE_ITER;
    }

    // Else, evict
    return CACHE_EXT_EVICT_NODE;
}

static void evict_main(struct cache_ext_eviction_ctx *eviction_ctx, struct mem_cgroup *memcg)
{
    /*
     * Iterate from head. If freq > 0, move to tail, freq--.
     * Otherwise, evict. (When evicting, move to tail in the meantime).
     */

    struct sampling_options opts = {
        .sample_size = 10,
    };
```

```
    if (bpf_cache_ext_list_sample(memcg, main_list, bpf_s3fifo_score_main_fn, &opts,
                      eviction_ctx)) {
        bpf_printk("cache_ext: evict: Failed to sample main_list\n");
        return;
    }

    // if (__sync_sub_and_fetch(&main_list_size, eviction_ctx->nr_folios_to_evict) < 0)
    //  main_list_size = 0;
}

#define MAIN_ITER_FN(id)                                \
static int bpf_s3fifo_score_main_iter_fn_##id(int idx, struct cache_ext_list_node *a)   \
{                                       \
    if (!folio_test_uptodate(a->folio) || !folio_test_lru(a->folio))        \
        return CACHE_EXT_CONTINUE_ITER;                     \
                                        \
    if (folio_test_dirty(a->folio) || folio_test_writeback(a->folio))       \
        return CACHE_EXT_CONTINUE_ITER;                     \
                                        \
    struct folio_metadata *data = get_folio_metadata(a->folio);             \
    if (!data) {                                \
        bpf_printk("cache_ext: score_fn: Failed to get metadata\n");        \
        return CACHE_EXT_CONTINUE_ITER;                     \
    }                                   \
                                        \
    s64 freq = __sync_sub_and_fetch(&data->freq, 1);                \
    if (freq < id) {                                \
        /*data->freq = 0;*/                             \
        return CACHE_EXT_EVICT_NODE;                        \
    }                                   \
                                        \
    return CACHE_EXT_CONTINUE_ITER;                         \
}

MAIN_ITER_FN(0)
MAIN_ITER_FN(1)
MAIN_ITER_FN(2)
MAIN_ITER_FN(3)

static void evict_main_iter(struct cache_ext_eviction_ctx *eviction_ctx, struct mem_cgroup *memcg)
{
    /*
     * Iterate from head. If freq > 0, move to tail, freq--.
     * Otherwise, evict. (When evicting, move to tail in the meantime).
     */

    struct cache_ext_iterate_opts opts = {
        .continue_list = CACHE_EXT_ITERATE_SELF,
        .continue_mode = CACHE_EXT_ITERATE_TAIL,
        .evict_list = CACHE_EXT_ITERATE_SELF,
        .evict_mode = CACHE_EXT_ITERATE_TAIL,
    };

    if (bpf_cache_ext_list_iterate_extended(memcg, main_list, bpf_s3fifo_score_main_iter_fn_0, &opts,
                        eviction_ctx) < 0) {
        bpf_printk("cache_ext: evict: Failed to iterate main_list\n");
        return;
    }

    if (eviction_ctx->nr_folios_to_evict < eviction_ctx->request_nr_folios_to_evict) {
        if (bpf_cache_ext_list_iterate_extended(memcg, main_list, bpf_s3fifo_score_main_iter_fn_1, &opts,
                            eviction_ctx) < 0) {
            bpf_printk("cache_ext: evict: Failed to iterate main_list\n");
            return;
        }
    } else {
        return;
    }

    if (eviction_ctx->nr_folios_to_evict < eviction_ctx->request_nr_folios_to_evict) {
        if (bpf_cache_ext_list_iterate_extended(memcg, main_list, bpf_s3fifo_score_main_iter_fn_2, &opts,
                            eviction_ctx) < 0) {
            bpf_printk("cache_ext: evict: Failed to iterate main_list\n");
            return;
        }
    } else {
        return;
    }

    if (eviction_ctx->nr_folios_to_evict < eviction_ctx->request_nr_folios_to_evict) {
        if (bpf_cache_ext_list_iterate_extended(memcg, main_list, bpf_s3fifo_score_main_iter_fn_3, &opts,
                            eviction_ctx) < 0) {
            bpf_printk("cache_ext: evict: Failed to iterate main_list\n");
            return;
        }
    }
}

static void evict_small(struct cache_ext_eviction_ctx *eviction_ctx, struct mem_cgroup *memcg)
{
    /*
     * Iterate from head. If freq > 1, move to main list, otherwise evict.
     * (When evicting, move to tail in the meantime).
     *
     * Use the iterate interface.
     */

    struct cache_ext_iterate_opts opts = {
        .continue_list = main_list,
        .continue_mode = CACHE_EXT_ITERATE_TAIL,
        .evict_list = CACHE_EXT_ITERATE_SELF,
        .evict_mode = CACHE_EXT_ITERATE_TAIL,
    };

    if (bpf_cache_ext_list_iterate_extended(memcg, small_list, bpf_s3fifo_score_small_fn, &opts,
                            eviction_ctx) < 0) {
        bpf_printk("cache_ext: evict: Failed to iterate small_list\n");
        return;
    }

    if (__sync_fetch_and_sub(&small_list_size, opts.nr_folios_continue) < 0)
        small_list_size = 0;

    if (__sync_fetch_and_add(&main_list_size, opts.nr_folios_continue) < 0)
        main_list_size = opts.nr_folios_continue;
}

void BPF_STRUCT_OPS(evo_policy_evict_folios, struct cache_ext_eviction_ctx *eviction_ctx,
            struct mem_cgroup *memcg)
{
    // bpf_printk("cache_ext: evict_folios: main_list_size: %lld, small_list_size: %lld, cache_size: %lld\n",
    //     main_list_size, small_list_size, cache_size);
    if (small_list_size >= cache_size / 15 || main_list_size <= 2 * small_list_size)
        evict_small(eviction_ctx, memcg);
    else
        evict_main_iter(eviction_ctx, memcg);
}

void BPF_STRUCT_OPS(evo_policy_folio_accessed, struct folio *folio) {
    if (!is_folio_relevant(folio))
        return;

    struct folio_metadata *data = get_folio_metadata(folio);
    if (!data) {
        bpf_printk("cache_ext: accessed: Failed to get metadata\n");
        return;
    }

    // Cap frequency at 3
    if (__sync_add_and_fetch(&data->freq, 1) > 3)
        data->freq = 3;
}

void BPF_STRUCT_OPS(evo_policy_folio_evicted, struct folio *folio) {
    u64 key = (u64)folio;
    u8 ghost_val = 0;

    // if (bpf_cache_ext_list_del(folio)) {
    //  bpf_printk("cache_ext: Failed to delete folio from sampling_list\n");
    //  return;
    // }

    struct ghost_entry ghost_key = {
        .address_space = (u64)folio->mapping->host,
        .offset = folio->index,
    };

    // Don't return early, we want to delete the folio metadata regardless
    if (bpf_map_update_elem(&ghost_map, &ghost_key, &ghost_val, BPF_ANY))
        bpf_printk("cache_ext: evicted: Failed to add to ghost_map\n");

    struct folio_metadata *data = get_folio_metadata(folio);
    if (!data) {
        //bpf_printk("cache_ext: evicted: Failed to get metadata\n");
        return;
    }

    if (data->in_main)
        __sync_fetch_and_sub(&main_list_size, 1);
    else
        __sync_fetch_and_sub(&small_list_size, 1);
```

```
    bpf_map_delete_elem(&folio_metadata_map, &key);

    // if (bpf_map_delete_elem(&folio_metadata_map, &key))
    //  bpf_printk("cache_ext: evicted: Failed to delete metadata\n");
}

/*
 * If folio is in the ghost map, add to tail of main list, otherwise add to tail
 * of small list.
 */
void BPF_STRUCT_OPS(evo_policy_folio_added, struct folio *folio) {
    if (!is_folio_relevant(folio))
        return;

    u64 key = (u64)folio;
    struct folio_metadata new_meta = {
        .freq = 0,
    };

    u64 list_to_add;
    if (folio_in_ghost(folio)) {
        // Peak test: ghost hits re-enter probation, but one access promotes them.
        list_to_add = small_list;
        new_meta.in_main = false;
        new_meta.freq = 1;
        __sync_fetch_and_add(&small_list_size, 1);
    } else {
        list_to_add = small_list;
        new_meta.in_main = false;
        __sync_fetch_and_add(&small_list_size, 1);
    }

    if (bpf_cache_ext_list_add_tail(list_to_add, folio)) {
        // TODO: add back to ghost_map?
        bpf_printk("cache_ext: added: Failed to add folio to main_list\n");
        return;
    }

    if (bpf_map_update_elem(&folio_metadata_map, &key, &new_meta, BPF_ANY)) {
        // TODO: add back to ghost_map? + error check delete call?
        bpf_cache_ext_list_del(folio);
        bpf_printk("cache_ext: added: Failed to create folio metadata\n");
        return;
    }
}

SEC(".struct_ops.link")
struct cache_ext_ops evo_policy_ops = {
    .init = (void *)evo_policy_init,
    .evict_folios = (void *)evo_policy_evict_folios,
    .folio_accessed = (void *)evo_policy_folio_accessed,
    .folio_evicted = (void *)evo_policy_folio_evicted,
    .folio_added = (void *)evo_policy_folio_added,
};
// EVOLVE-BLOCK-END

// ============================================================================
// SECTION: USERSPACE LOADER
// ============================================================================
// Loader for the S3-FIFO policy: parses -w/-s/-c like every policy in this repo,
// attaches the struct_ops to the cgroup and waits for SIGINT/SIGTERM.
// EVOLVE-BLOCK-START
#include <argp.h>
#include <errno.h>
#include <stdlib.h>
#include <bpf/bpf.h>
#include <fcntl.h>
#include <limits.h>
#include <signal.h>
#include <stdint.h>
#include <stdio.h>
#include <string.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <unistd.h>

#include "dir_watcher.h"
#include "evo_policy.skel.h"

char *USAGE = "Usage: ./evo_policy --watch_dir <dir> --cgroup_size <size> --cgroup_path <path>\n";
struct cmdline_args {
    char *watch_dir;
        uint64_t cgroup_size;
        char *cgroup_path;
};

static struct argp_option options[] = {
    { "watch_dir", 'w', "DIR", 0, "Directory to watch" },
        {"cgroup_size", 's', "SIZE", 0, "Size of the cgroup"},
        {"cgroup_path", 'c', "PATH", 0, "Path to cgroup (e.g., /sys/fs/cgroup/cache_ext_test)"},
    { 0 },
};

static const uint64_t page_size = 4096;

static volatile sig_atomic_t exiting;

static void sig_handler(int signo) {
    exiting = 1;
}

static error_t parse_opt(int key, char *arg, struct argp_state *state)
{
    struct cmdline_args *args = state->input;
    switch (key) {
    case 'w':
        args->watch_dir = arg;
        break;
        case 's':
                // TODO: move this to parse_args()
                errno = 0;
                args->cgroup_size = strtoull(arg, NULL, 10);
                if (errno)
                        args->cgroup_size = 0;

                break;
        case 'c':
                args->cgroup_path = arg;
                break;
    default:
        return ARGP_ERR_UNKNOWN;
    }
    return 0;
}

static int parse_args(int argc, char **argv, struct cmdline_args *args) {
    struct argp argp = { options, parse_opt, 0, 0 };
    argp_parse(&argp, argc, argv, 0, 0, args);

    if (args->watch_dir == NULL) {
        fprintf(stderr, "Missing required argument: watch_dir\n");
        return 1;
    }

    if (args->cgroup_size == 0) {
            fprintf(stderr, "Invalid cgroup size\n");
            return 1;
    }

    if (args->cgroup_path == NULL) {
        fprintf(stderr, "Missing required argument: cgroup_path\n");
        return 1;
    }

    return 0;
}

/*
 * Validate watch_dir
 *
 * watch_dir_full_path must be able to hold PATH_MAX bytes.
 */
static int validate_watch_dir(const char *watch_dir, char *watch_dir_full_path) {
    // Does watch_dir exist?
    if (access(watch_dir, F_OK) == -1) {
        fprintf(stderr, "Directory does not exist: %s\n", watch_dir);
        return 1;
    }

    // Get full path of watch_dir
    if (realpath(watch_dir, watch_dir_full_path) == NULL) {
        perror("realpath");
        return 1;
    }

    // BPF policy restriction
    if (strlen(watch_dir_full_path) > 128) {
        fprintf(stderr, "watch_dir path too long\n");
        return 1;
    }
```

```
    return 0;
}

int main(int argc, char **argv) {
    struct cmdline_args args = { 0 };
    struct evo_policy_bpf *skel = NULL;
    struct bpf_link *link = NULL;
    struct sigaction sa;
    char watch_dir_path[PATH_MAX];
    int cgroup_fd = -1;
    int ret = 1;

    libbpf_set_strict_mode(LIBBPF_STRICT_ALL);

    if (parse_args(argc, argv, &args))
        return 1;

    memset(&sa, 0, sizeof(sa));
    sigemptyset(&sa.sa_mask);
    sa.sa_handler = sig_handler;

    // Install signal handler
    if (sigaction(SIGINT, &sa, NULL) || sigaction(SIGTERM, &sa, NULL)) {
        perror("Failed to set up signal handling");
        return 1;
    }

    if (validate_watch_dir(args.watch_dir, watch_dir_path))
        return 1;

    // Open cgroup directory early
    cgroup_fd = open(args.cgroup_path, O_RDONLY);
    if (cgroup_fd < 0) {
        perror("Failed to open cgroup path");
        return 1;
    }

    skel = evo_policy_bpf__open();
    if (!skel) {
        perror("Failed to open BPF skeleton");
        goto cleanup;
    }

    // Set cache size in terms of number of pages. Assumes uniform page size.
    skel->rodata->cache_size = args.cgroup_size / page_size;
    fprintf(stderr, "Cgroup size: %lu bytes\n", args.cgroup_size);
    fprintf(stderr, "Cache size: %lu pages\n", skel->rodata->cache_size);

    // Resize ghost_map
    if (bpf_map__set_max_entries(skel->maps.ghost_map, skel->rodata->cache_size)) {
        perror("Failed to resize ghost_map");
        ret = 1;
        goto cleanup;
    }

    // Set watch_dir
    watch_dir_path_len_map(skel) = strlen(watch_dir_path);
    strcpy(watch_dir_path_map(skel), watch_dir_path);

    if (evo_policy_bpf__load(skel)) {
        perror("Failed to load BPF skeleton");
        ret = 1;
        goto cleanup;
    }

    if (initialize_watch_dir_map(watch_dir_path, bpf_map__fd(inode_watchlist_map(skel)), true)) {
        perror("Failed to initialize watch_dir map");
        ret = 1;
        goto cleanup;
    }

    link = bpf_map__attach_cache_ext_ops(skel->maps.evo_policy_ops, cgroup_fd);
    if (link == NULL) {
        perror("Failed to attach cache_ext_ops to cgroup");
        ret = 1;
        goto cleanup;
    }

    // This is necessary for the dir_watcher functionality
    if (evo_policy_bpf__attach(skel)) {
        perror("Failed to attach BPF skeleton");
        ret = 1;
        goto cleanup;
    }

    printf("evo_policy (S3-FIFO) running. Press Ctrl+C to exit...\n");
    fflush(stdout);
    while (!exiting)
        sleep(1);
    ret = 0;

cleanup:
    close(cgroup_fd);
    bpf_link__destroy(link);
    evo_policy_bpf__destroy(skel);
    return ret;
}
// EVOLVE-BLOCK-END
```

Notes and caveats

- Hardware: pc309.emulab.net is a 2-core ~2 GB node; absolute numbers differ from other hosts and from the earlier c220g1 runs.
- Repeated evaluations of identical code vary by a few percent; differences of a few ops/s (or a few hundredths of tanh score) between attempts are within noise.
- No tune attempts were used.
- Best-by-score is a single measurement; re-measure before drawing conclusions about small gaps.